

static block OR static initializer :

A static block is a special block in java which is automatically executed at the time of **loading the .class file into JVM memory**.

Example :

```
public class A
{
    static
    {
        //static block   [Whenever A.class file will be loaded then static block will be executed]
    }
}
```

* In Java, A class will be loaded as per request and a class will be loaded only **one time hence static block will be executed only one time**.

* A static block is executed **before main method** that is the reason we cannot write static block inside the main method OR any static method

* The main purpose of static block to initialize the **static data member of the class that is the reason it is also known as Static Initializer**.

* If a class contains multiple static blocks then all these blocks will be executed from top to bottom.

* Just like non static block, We **cannot write** return statement inside static block.

* A static blank final field **must be initialized inside static block only**.

Example :

```
class Test
{
    static final int A; //Static blank final field

    static
    {
        A = 900;
        IO.println(A);
    }
}
```

* A static blank final field also have default value provided by prepare phase.

* If we don't declare static field before static block body execution (post declaration) then we can perform write operation(Initialization is possible due to prepare phase) but read operation is not possible DIRECTLY otherwise we will get an error Illegal forward reference, It is possible with class name because now compiler knows that variable is coming from class area OR Method area.

Example :

```
static
{
    x = 100; //Valid
    IO.println(x); //Invalid
    IO.println(ClassName.x); //valid
}
static int x; //Post Declaration
```

We can directly perform read and write operation inside a static method then it is valid operation because static method is having low priority than static block.

```
public static void print()
{
    y = 200; //Valid
    IO.println(y); //Valid [Here class name is not reqd]
}
static int y; //Post Declaration
```

//Programs of static block :

```
//static block
class Foo
{
    Foo()
    {
        IO.println("No Argument constructor..");
    }

    {
        IO.println("Non static Block..");
    }

    static
    {
        IO.println("Static block...");
    }
}

public class StaticBlockDemo
{
    public static void main(String []args)
    {
        IO.println("Main Method Executed ");
    }
}
```

Here Foo.class file is not loaded into JVM Memory so static block of Foo class will not be executed.

```
class Test
{
    static int x;

    static
    {
        IO.println("x value is :"+x);
    }

    static
    {
        x = 200;
        IO.println("x value is :"+x);
    }

    static
    {
        x = 300;
        IO.println("x value is :"+x);
    }
}

public class StaticBlockDemo1
{
    void main()
    {
        IO.println("Main Method");
        IO.println("From main method :"+Test.x); //Here Test.class will be loaded first
    }
}
```

Note : If a class contains more than 1 static block then it will be executed from top to bottom.

```
class Foo
{
    static int x;

    static
    {
        IO.println("x value is :"+x);
    }
}

public class StaticBlockDemo2
{
    void main()
    {
        new Foo(); //Here Foo.class file will be loaded first
    }
}
```

Note : static variables are also having default value provided by prepare phase.

```
class Demo
{
    final static int a ; //static Blank final field

    static
    {
        print();
        a = 100;
        IO.println("User Value :"+a);
    }

    public static void print()
    {
        IO.println("Default Value :"+a);
    }
}

public class StaticBlockDemo3
{
    void main()
    {
        new Demo(); //Demo.class will be loaded here
    }
}
```

A static blank final field must be initialized by static block only and It also have default value.

```
class Alpha
{
    static
    {
        IO.println("Alpha class static block");
    }
}

class Beta extends Alpha //Inheritance [Alpha-super, Beta-Sub]
{
    static
    {
        IO.println("Beta class static block");
    }
}
```

```
public class StaticBlockDemo4
{
    void main()
    {
        new Beta(); //Try to load Beta class
    }
}
```

In class loading phase, First of all super class will be loaded then only sub class will be loaded.

-